CHAPTER 8

IMPLEMENTATION

8.1 Introduction

This basketball project involves implementing an automated system to detect and track players during basketball game videos using computer vision and deep learning techniques. The implementation begins with collecting and preprocessing video data from basketball matches. Frames are extracted and fed into a Convolutional Neural Network (CNN) trained to recognize and identify individual players on the court.

To accurately track players across frames, the system uses object tracking algorithms that associate detected players over time, handling challenges such as player occlusion and movement speed. Additionally, a semi-automatic algorithm is implemented to detect and map the basketball court, which helps in positioning players accurately in the spatial context of the game.

The system is built using Python and popular libraries such as OpenCV for video processing, TensorFlow/PyTorch for implementing CNN models, and tracking algorithms like SORT (Simple Online and Realtime Tracking) or Deep SORT for multi-object tracking. The output includes real-time visualizations of player trajectories and analytical data to support coaching decisions.

Through this implementation, the project demonstrates how machine learning and computer vision can be effectively applied to sports analytics, providing an automated, scalable, and efficient solution for basketball player detection and tracking.

8.2 Tools and Technologies Used

1. **Python:** The primary programming language due to its extensive libraries and ease of use for machine learning and computer vision tasks.
2. **OpenCV:** An open-source computer vision library used for video processing, frame extraction, image manipulation, and applying various image processing algorithms.
3. **TensorFlow / PyTorch:** Deep learning frameworks used to design, train, and deploy Convolutional Neural Networks (CNNs) for player detection and classification.
4. **Pre-trained CNN Models:** Models like YOLO (You Only Look Once), Faster R-CNN, or MobileNet were utilized or fine-tuned for efficient and accurate player detection.
5. **Object Tracking Algorithms:** Algorithms such as SORT (Simple Online and Realtime Tracking) or Deep SORT were used for tracking multiple players across video frames.

6. **NumPy and Pandas:** Libraries for numerical computations and data handling, used for processing and analyzing player position data.
7. **Matplotlib / Seaborn:** Visualization libraries to plot player trajectories and movement heatmaps for performance analysis.
8. **Jupyter Notebook / IDEs (PyCharm, VS Code):** Development environments used for coding, testing, and debugging the project.
9. **Hardware:** A computer with a good CPU/GPU configuration (e.g., NVIDIA GPU) to efficiently train deep learning models and process video data.
10. **Video Datasets:** Basketball game footage sourced from publicly available datasets or recorded videos for training and testing the system.

8.3 Methodologies / Algorithm Details

8.3.1 You Only Look Once (YOLO) Algorithm

YOLO is a deep learning–based real-time object detection algorithm that detects multiple objects in images or videos with high speed and decent accuracy. It belongs to the family of single-shot detectors and is widely used in applications such as surveillance, autonomous driving, and sports analytics.

Methodology of YOLO

YOLO reformulates the detection process as a single regression problem, making it extremely fast and efficient for real-time applications. Instead of multiple stages like region proposals and classification, YOLO processes the entire input image in a single forward pass through a convolutional neural network.

The image is divided into a grid, and each grid cell predicts bounding boxes, objectness scores, and class probabilities. This unified architecture allows YOLO to simultaneously detect multiple objects (e.g., basketball players) with high speed and reasonable accuracy. For basketball player detection, YOLO enables real-time identification and localization on the court, suitable for sports analytics where speed and performance are critical.

Algorithm Steps

- **Input Preprocessing:** Resize input images to fixed dimensions (e.g.,640×640 pixels) and normalize pixel values before feeding into the network.
- **Grid Division:** Divide the image into an $S \times S$ grid (e.g., 13×13). Each cell is responsible for detecting objects whose centers fall within it.
- **Prediction per Cell:** Each grid cell predicts multiple bounding boxes (B boxes), confidence scores (objectness), and class probabilities (C classes).
- **Bounding Box Adjustment:** Coordinates are adjusted using sigmoid functions and scaled with anchor box references for accurate localization.
- **Score Calculation:** Confidence scores are multiplied by class probabilities to rank detections.
- **Thresholding:** Apply a confidence threshold to filter out low-confidence detections.
- **Non-Maximum Suppression (NMS):** Eliminate overlapping boxes for the same object, retaining the highest confidence box.
- **Final Output:** Remaining bounding boxes with class labels and confidence scores are the detected objects ready for visualization or further processing.

8.3.2 OpenPose

OpenPose is a real-time multi-person keypoint detection library for body, face, hands, and foot estimation. It uses deep learning to detect human poses by identifying key joints and connecting them into skeletal structures. It is widely used in sports analytics, fitness tracking, and motion understanding.

Methodology

OpenPose uses a bottom-up approach: first detecting body parts (keypoints) and then associating them with individuals using Part Affinity Fields (PAFs). A multi-stage CNN predicts confidence maps for joints and PAFs that describe associations between body parts.

The network outputs 2D maps indicating the likelihood of each body part at each pixel, then connects parts by finding the most likely paths per the PAFs, building skeletons for each person. This enables simultaneous multi-person pose estimation, ideal for sports like basketball.

Algorithm Steps

- **Input Image Preprocessing:** Resize, normalize, and sometimes pad input images; perform mean subtraction to improve detection.
- **Feature Extraction:** CNN (e.g., based on VGG or ResNet) extracts low and mid-level features retaining spatial structure.
- **Confidence Map Prediction:** For each keypoint (e.g., nose, shoulder), generate a confidence map indicating probability of presence.
- **Part Affinity Fields (PAFs):** Predict 2D vector fields encoding orientation and association between keypoints.
- **Keypoint Detection:** Detect local peaks in confidence maps; apply non-maximum suppression to reduce duplicates.
- **Keypoint Association:** Use PAFs to connect keypoints into full skeletons for each detected person.
- **Pose Estimation Output:** Output is a set of skeletons with 2D joint coordinates, useful for visualization or further analysis.

8.3.3 Convolutional Neural Networks (CNN)

CNNs are deep learning algorithms highly effective for visual imagery analysis, including image classification, object detection, and pattern recognition — ideal for sports video analysis such as basketball player detection.

Methodology

CNNs automatically extract spatial hierarchies of features through multiple layers like convolution, pooling, and fully connected layers. Early layers detect simple features (edges, corners), while deeper layers capture complex patterns (textures, shapes, objects).

The model is trained on large datasets using backpropagation to minimize errors. Once trained, CNNs can classify or detect features in new images based on learned patterns, capturing spatial and positional information effectively.

Algorithm:

1. **Input Layer**: The input image (e.g., 224×224×3 for RGB) is passed into the CNN.

Each pixel's RGB values are normalized to help the model converge faster during training.

2. **Convolution Layer**: Applies multiple learnable filters (kernels) sliding over the image to produce feature maps highlighting edges, textures, or patterns.
3. **Activation Function (ReLU)**: Introduces non-linearity by removing negative values and speeding up training.
4. **Pooling Layer**: Downsamples feature maps (usually max pooling) to reduce spatial dimensions, computational cost, and overfitting.
5. **Deeper Convolution + Pooling Layers**: Repeat convolution and pooling with more filters to learn deeper features like shapes, body parts, or jersey numbers.
6. **Flattening**: Converts the final pooling output into a one-dimensional vector for classification.
7. **Fully Connected (Dense) Layer**: Performs classification by combining all features and assigning probabilities to each class (e.g., player, ball, court).
8. **Output Layer (Softmax/Sigmoid)**: Uses softmax (for multi-class) or sigmoid (for binary) to produce class probability scores. The class with the highest score is selected as the prediction.

8.4 Screenshots

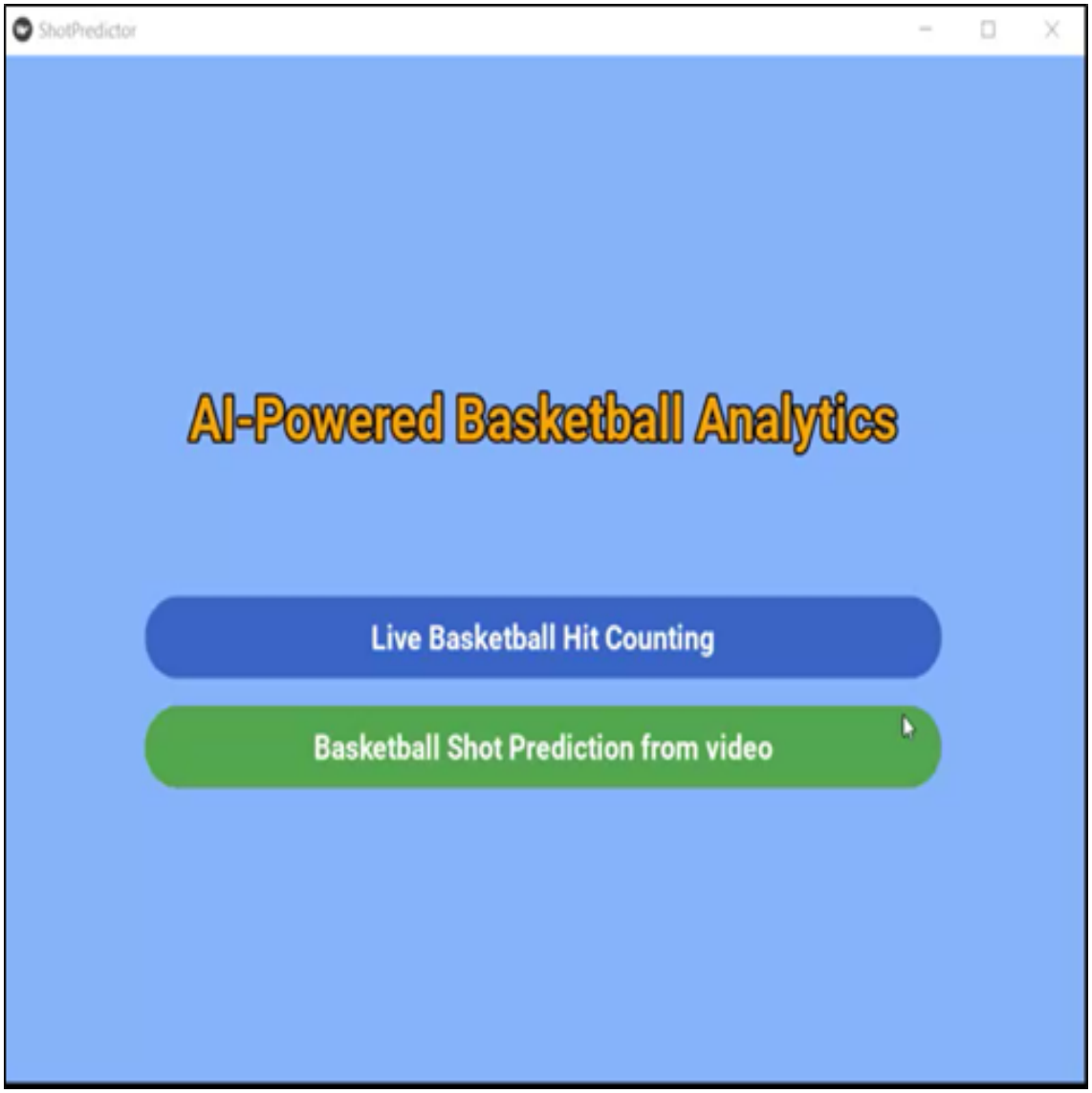

Figure 8.1: AI Basketball Analytics Menu

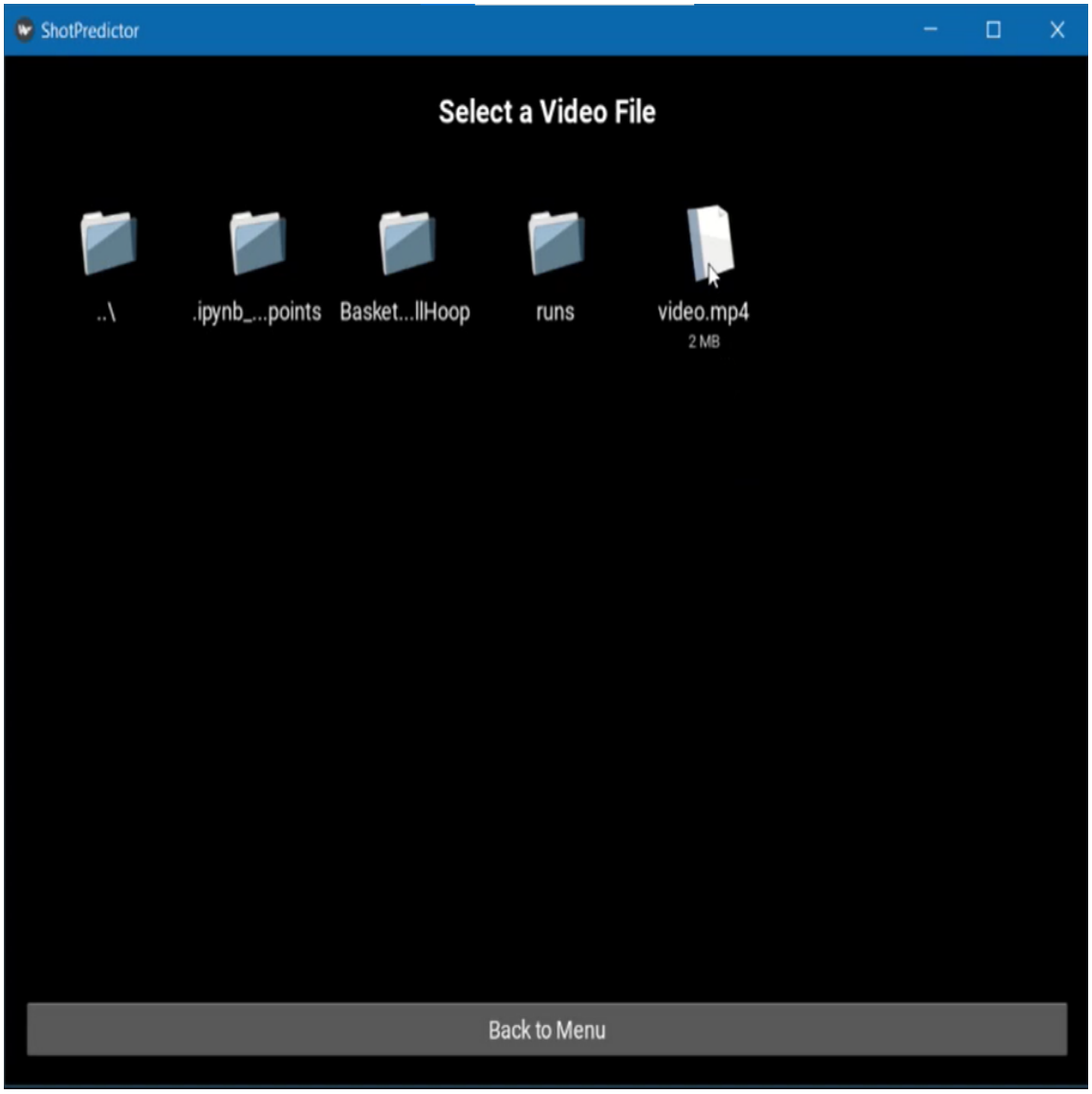

Figure 8.2: Video File Selection

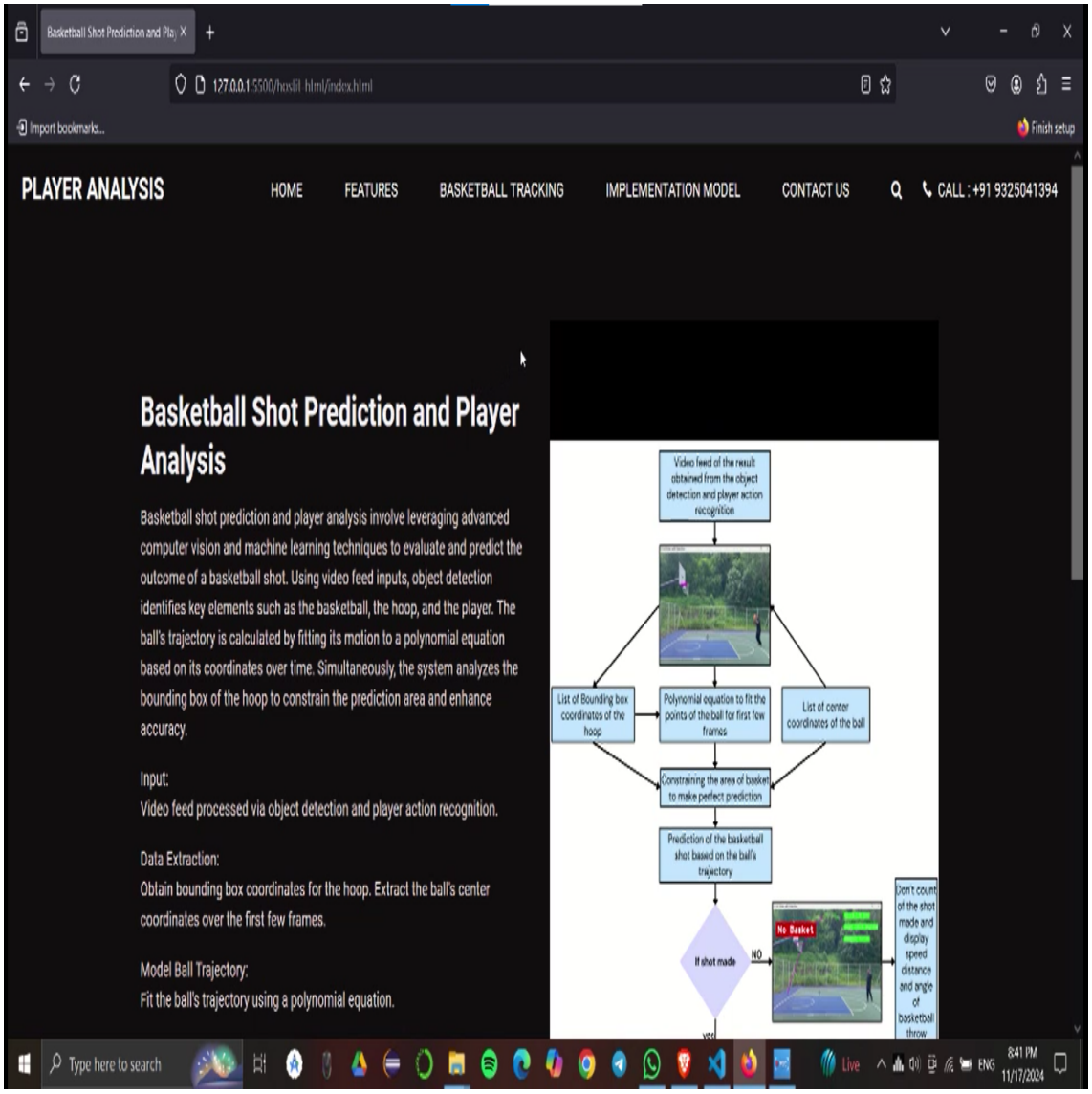

Figure 8.3: Web-based Shot Analysis Platform

Figure 8.4: Basketball Vision Analysis Overview

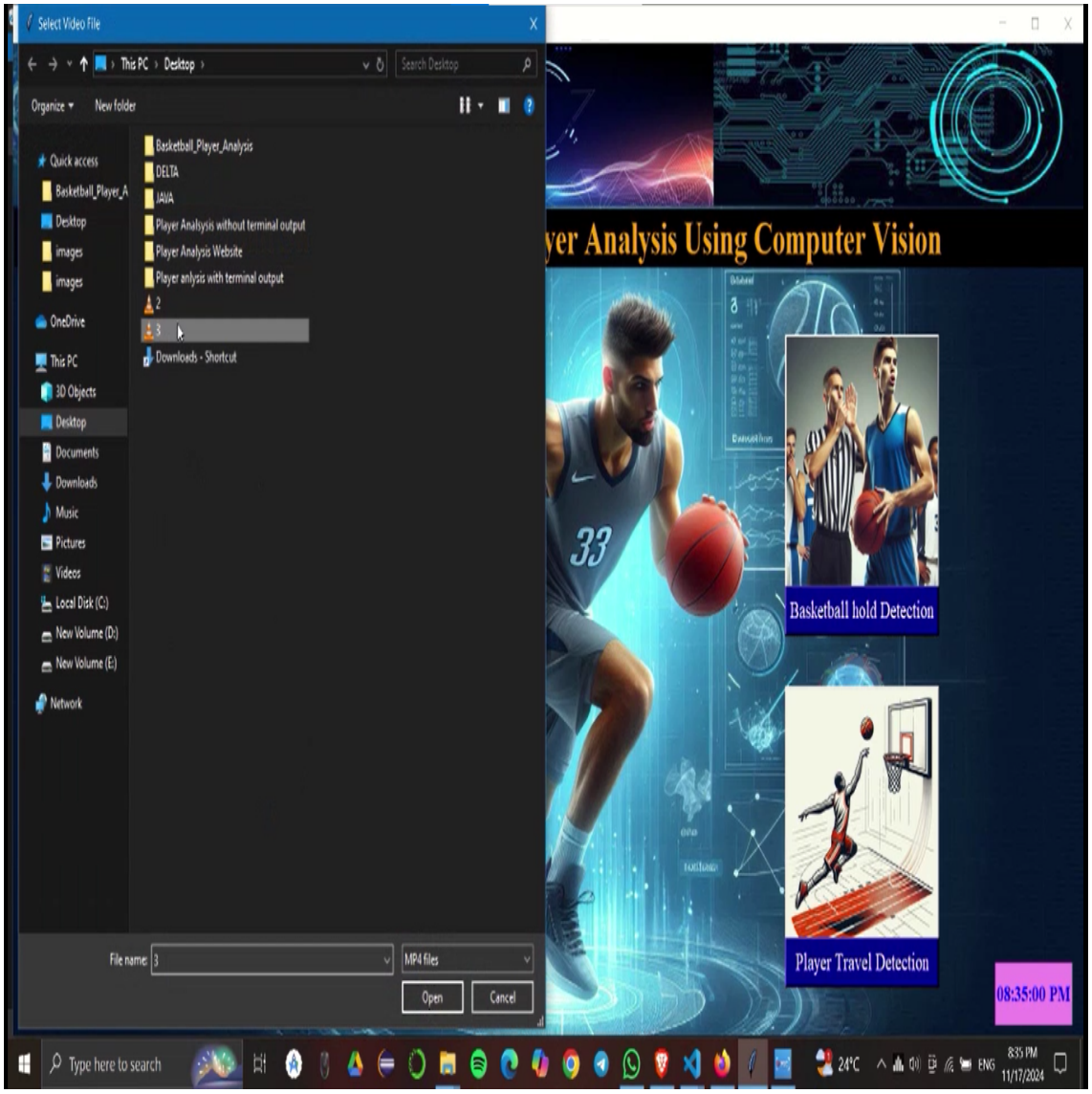

Figure 8.5: Player Analysis File Open